```
// Multi-year ROI projection calculations

export interface SingleYearResults {
  annualBenefits: number;
  annualCosts: number;
  timeSavings: number;
  qualitySavings: number;
  revenueSavings: number;
  hoursSavedWeekly: number;
  hoursSavedAnnually: number;
  ftesSaved: number;
  errorReductionPercent: number;
  errorsAvoidedAnnually: number;
  hoursReallocatedAnnually: number;
  [key: string]: number;
}

export interface CostBreakdown {
  implementationCost: number;
  annualSoftwareCost: number;
  trainingCosts: number;
  changeMgmtCosts: number;
}

export interface YearlyProjection {
  year: number;
  benefits: number;
  costs: number;
  netBenefit: number;
  cumulativeBenefits: number;
  cumulativeCosts: number;
  cumulativeNetBenefit: number;
  roi: number;
  cumulativeROI: number;
  timeSavings: number;
  qualitySavings: number;
  revenueSavings: number;
}

export interface MultiYearResults {
  yearlyProjections: YearlyProjection[];
  totalBenefits5Year: number;
  totalCosts5Year: number;
  netBenefit5Year: number;
```

```
  roi5Year: number;
  paybackMonths: number;
  year1RampFactor: number;
}

/**
 * Calculate benefit multiplier for a given year based on ramp-up period
 * Handles ramp-up periods >12 months correctly
 *
 * Business Rule: During ramp-up months, assume 50% average efficiency.
 * After ramp-up completes, assume 100% efficiency.
 *
 * @param year - The year number (1, 2, 3, etc.)
 * @param rampUpMonths - Total ramp-up period in months
 * @param efficiencyAdjustedRampFactor - Optional pre-calculated Year 1 ramp factor with
efficiency applied
 * @returns Benefit multiplier for that year (0.0 to 1.0)
 */
function calculateYearBenefitMultiplier(
  year: number,
  rampUpMonths: number,
  efficiencyAdjustedRampFactor?: number
): number {
  if (rampUpMonths <= 12) {
    // Ramp-up completes within Year 1
    // Formula: (full production months + 50% during ramp-up months) / 12
    if (year === 1) {
      return efficiencyAdjustedRampFactor !== undefined
        ? efficiencyAdjustedRampFactor
        : (12 - rampUpMonths + (rampUpMonths * 0.5)) / 12;
    }
    // Years 2+ get full benefits
    return 1.0;
  } else {
    // Ramp-up extends beyond Year 1
    const yearStartMonth = (year - 1) * 12 + 1; // First month of this year (1-indexed)
    const yearEndMonth = year * 12; // Last month of this year

    // Determine how many months of this year are still ramping up
    const lastRampMonth = rampUpMonths; // Ramp-up completes at end of this month

    if (yearStartMonth > lastRampMonth) {
      // This year starts after ramp-up completed - full 100% benefits
      return 1.0;
```

```
    }

    // Calculate how many months of this year are still ramping up
    const monthsStillRamping = Math.min(12, Math.max(0, lastRampMonth - yearStartMonth +
1));
    const monthsAtFullCapacity = 12 - monthsStillRamping;

    if (monthsAtFullCapacity === 0) {
      // Entire year is still within ramp-up period - 50% average efficiency
      if (year === 1 && efficiencyAdjustedRampFactor !== undefined) {
        // Year 1 uses efficiency-adjusted factor if provided
        return efficiencyAdjustedRampFactor;
      }
      return 0.5;
    }

    // Ramp-up completes partway through this year
    // Weighted average: ramp-up months at 50%, full production months at 100%
    const yearAvg = (monthsStillRamping * 0.5 + monthsAtFullCapacity * 1.0) / 12;

    return yearAvg;
  }
}

/**
 * Calculate multi-year ROI projections with ramp-up modeling
 *
 * @param baseResults - Single year calculation results (full run-rate)
 * @param costBreakdown - Detailed cost breakdown for multi-year modeling
 * @param rampUpMonths - Implementation ramp-up period (1-36 months supported)
 * @param years - Number of years to project (default: 5)
 * @param efficiencyAdjustedRampFactor - Optional pre-calculated ramp factor with efficiency
applied (for Year 1 only)
 * @returns Multi-year projection results
 */
export function calculateMultiYearROI(
  baseResults: SingleYearResults,
  costBreakdown: CostBreakdown,
  rampUpMonths: number = 6,
  years: number = 5,
  efficiencyAdjustedRampFactor?: number
): MultiYearResults {
  const yearlyProjections: YearlyProjection[] = [];
```

```
  // Year 1 ramp-up factor using the correct calculation for all ramp-up periods
  // This ensures consistency between reported factor and actual benefits calculation
  const year1RampFactor = calculateYearBenefitMultiplier(1, rampUpMonths,
efficiencyAdjustedRampFactor);

  console.log('🔧 Multi-Year Calculation:', {
    rampUpMonths,
    year1RampFactor: `${(year1RampFactor * 100).toFixed(1)}%`,
    fullAnnualBenefits: baseResults.annualBenefits,
    year1Benefits: baseResults.annualBenefits * year1RampFactor,
    year1Costs: costBreakdown.implementationCost + costBreakdown.annualSoftwareCost +
costBreakdown.trainingCosts + costBreakdown.changeMgmtCosts
  });

  let cumulativeBenefits = 0;
  let cumulativeCosts = 0;
  let paybackMonths = 0;
  let paybackFound = false;

  for (let year = 1; year <= years; year++) {
    // Benefits calculation using helper function that handles >12 month ramp-ups
    const benefitMultiplier = calculateYearBenefitMultiplier(year, rampUpMonths,
efficiencyAdjustedRampFactor);
    const yearBenefits = baseResults.annualBenefits * benefitMultiplier;
    const yearTimeSavings = baseResults.timeSavings * benefitMultiplier;
    const yearQualitySavings = baseResults.qualitySavings * benefitMultiplier;
    const yearRevenueSavings = baseResults.revenueSavings * benefitMultiplier;

    // Costs calculation
    let yearCosts = costBreakdown.annualSoftwareCost; // Recurring software cost every year

    if (year === 1) {
      // Year 1: One-time implementation, training, and change management costs
      yearCosts += costBreakdown.implementationCost +
                   costBreakdown.trainingCosts +
                   costBreakdown.changeMgmtCosts;
    }

    // Calculate net benefit for the year
    const netBenefit = yearBenefits - yearCosts;

    // Calculate cumulative values
    cumulativeBenefits += yearBenefits;
    cumulativeCosts += yearCosts;
```

```
    const cumulativeNetBenefit = cumulativeBenefits - cumulativeCosts;

    // Calculate ROI for this year
    const roi = yearCosts > 0 ? (netBenefit / yearCosts) * 100 : 0;
    const cumulativeROI = cumulativeCosts > 0 ? (cumulativeNetBenefit / cumulativeCosts) * 100
: 0;

    // Calculate payback period (first time cumulative net benefit becomes positive)
    if (!paybackFound && cumulativeNetBenefit >= 0) {
      if (cumulativeNetBenefit === 0) {
        paybackMonths = year * 12;
      } else {
        // More precise calculation: interpolate within the year
        const prevCumulativeNet = year > 1
          ? yearlyProjections[year - 2].cumulativeNetBenefit
          : -yearCosts; // CRITICAL FIX: Year 1 starts with full costs as deficit

        // Monthly net benefit for this year
        const monthlyNet = netBenefit / 12;

        if (monthlyNet > 0 && prevCumulativeNet < 0) {
          // Interpolate to find exact month of breakeven
          const monthsToBreakeven = Math.abs(prevCumulativeNet) / monthlyNet;
          paybackMonths = ((year - 1) * 12) + monthsToBreakeven;
        } else {
          // Breakeven happens exactly at year boundary
          paybackMonths = year * 12;
        }
      }
      paybackFound = true;
    }

    yearlyProjections.push({
      year,
      benefits: yearBenefits,
      costs: yearCosts,
      netBenefit,
      cumulativeBenefits,
      cumulativeCosts,
      cumulativeNetBenefit,
      roi,
      cumulativeROI,
      timeSavings: yearTimeSavings,
      qualitySavings: yearQualitySavings,
```

```
      revenueSavings: yearRevenueSavings,
    });
  }

  // If payback never happens within the projection period
  if (!paybackFound) {
    const lastYear = yearlyProjections[yearlyProjections.length - 1];
    if (lastYear.cumulativeNetBenefit < 0) {
      paybackMonths = years * 12 + 12; // Beyond projection period
    }
  }

  const lastProjection = yearlyProjections[years - 1];

  return {
    yearlyProjections,
    totalBenefits5Year: lastProjection.cumulativeBenefits,
    totalCosts5Year: lastProjection.cumulativeCosts,
    netBenefit5Year: lastProjection.cumulativeNetBenefit,
    roi5Year: lastProjection.cumulativeROI,
    paybackMonths,
    year1RampFactor,
  };
}

/**
 * Get a specific year's projection
 */
export function getYearProjection(
  multiYearResults: MultiYearResults,
  year: number
): YearlyProjection | undefined {
  return multiYearResults.yearlyProjections.find(p => p.year === year);
}

/**
 * Format payback period in human-readable format
 */
export function formatPaybackPeriod(paybackMonths: number): string {
  if (paybackMonths <= 0) return 'Immediate';
  if (paybackMonths > 60) return 'Beyond 5 years';

  const years = Math.floor(paybackMonths / 12);
  const months = Math.round(paybackMonths % 12);
```

```
  if (years === 0) return `${months} month${months !== 1 ? 's' : ''}`;
  if (months === 0) return `${years} year${years !== 1 ? 's' : ''}`;
  return `${years} year${years !== 1 ? 's' : ''} ${months} month${months !== 1 ? 's' : ''}`;
}
```